

vesti

A transpiler that compiles into $\text{\LaTeX}$

The Full Reference Manual

Language grammar, keywords, builtins, the preprocessor,
and the embedded Lua scripting API

```
docclass article (10pt)
importpkg { amsmath, geometry (a4paper) }
startdoc
Hello, Vesti!
```

This manual is generated from a full reading of the vesti source tree (lexer, parser, preprocessor, code generator and the Lua bindings). It documents every keyword, builtin, preprocessor directive and Lua API function exposed by the compiler.

Status: beta — breaking changes may still occur.

Contents

1	Introduction	3
2	Installation and configuration	3
2.1	Building the compiler	3
2.2	The configuration file	3
2.3	L ^A T _E X engines	4
3	The command-line interface	4
3.1	<code>compile</code> — transpile and build	4
3.2	<code>latex</code> — build an existing <code>.tex</code> with Tectonic	4
3.3	<code>init</code> — scaffold a project	4
3.4	<code>clear</code> — remove build artefacts	5
3.5	<code>experimental</code>	5
4	Project structure and the build pipeline	5
5	Structure of a <code>vesti</code> file	5
6	Lexical grammar	5
6.1	Comments	5
6.2	Escapes and literal characters	6
6.3	Raw L ^A T _E X pass-through (verbatim)	6
6.4	Numbers, words and L ^A T _E X commands	6
6.5	Math symbol conversion	6
7	Keywords	7
7.1	<code>docclass</code> — document class	7
7.2	<code>importpkg</code> — load packages	7
7.3	<code>importves</code> — include another <code>vesti</code> file	7
7.4	<code>importmod</code> — fetch and use a <code>vesti</code> module	7
7.5	<code>startdoc</code> — begin the body	8
7.6	<code>useenv</code> — an environment	8
7.7	<code>begenv</code> / <code>endenv</code> — explicit pair	8
7.8	<code>defun</code> — define a command	8
7.9	<code>defenv</code> — define an environment	8
8	Math mode	9
8.1	Entering math mode	9
8.2	Fractions with <code>//</code>	9
8.3	Auto-sized delimiters with <code>?</code>	9
8.4	Text inside math with <code>"..."</code>	9

8.5	Mode-switching builtins	10
9	Builtins	10
9.1	#label	10
9.2	#eq	10
9.3	#chardef	10
9.4	#mathchardef	11
9.5	#enum and #enum_counter	11
9.6	#get_filepath	11
9.7	#copy_file	11
9.8	#picture	11
9.9	#showfont	11
9.10	#raw_tex	12
9.11	#engine_type	12
9.12	#textmode and #mathmode	12
10	The preprocessor	12
10.1	#def and #undef — token macros	12
10.2	#include — textual file inclusion	12
10.3	#at_on / #at_off — @ as a letter	12
10.4	#ltx3_on / #ltx3_off / #noltx3	13
10.5	Defun parameter tokens	13
11	Lua scripting	13
11.1	Inline Lua blocks	13
11.2	Block modifiers: global, import, export	13
11.3	Build scripts	14
11.4	vesti.compile — the build entry point	14
12	The Lua API reference	14
13	Deprecated features	15
14	A worked example	15
15	Appendix: quick reference	15

1 Introduction

Vesti is a transpiler: it reads a `.ves` source file and emits $\text{\LaTeX}$, which a $\text{\TeX}$ engine then turns into a PDF. The motivation is that raw $\text{\LaTeX}$ is verbose — especially the endless `\begin{...} / \end{...}` pairs — while its markup is uncomfortable to write. Vesti keeps the full power of $\text{\LaTeX}$ (you may drop into raw $\text{\LaTeX}$ at any point) and adds a small, readable layer of keywords, math shorthands, a macro preprocessor and an embedded Lua scripting engine.

The $\text{\LaTeX}$ on the left is produced by the vesti on the right:

```
\documentclass[tikz,geometry]{coprime}
\begin{document}
\section{Foo}
Hello, World!
\begin{figure}[ht]
  \centering
  \begin{tikzpicture}
    \draw (0,0) -- (1,1);
  \end{tikzpicture}
\end{figure}
\end{document}
```

```
docclass coprime (tikz, geometry)

startdoc
\section{Foo}
Hello, World!
useenv figure [ht] {
  \centering
  useenv tikzpicture {
    \draw (0,0) -- (1,1);
  }
}
```

Notice there is no closing `startdoc`: vesti ends the document automatically at end-of-file, since virtually no real document has material after `\end{document}`.

Conventions. Throughout this manual, **keywords** are shown in purple, **#builtins** in blue (always prefixed with #), and the symbol `□` marks a position where a space is mandatory. Code samples are set in framed verbatim blocks.

2 Installation and configuration

2.1 Building the compiler

Vesti is written in Zig (master branch) with an optional Rust-based Tectonic backend. For an ordinary install with Tectonic bundled in:

```
$ zig build --prefix-exe-dir <install-path> -Doptimize=ReleaseSafe
```

To build without the Tectonic backend (you then drive an external $\text{\TeX}$ engine yourself), add `-Dtectonic=false`. Building the Tectonic dynamic library additionally needs a Rust toolchain and the native libraries `fontconfig`, `freetype2`, `graphite2`, `harfbuzz`, `ICU4C` and `libpng`, plus the `upx` binary. On Linux the `zenity` dialog tool is also recommended.

2.2 The configuration file

Vesti reads an optional configuration file written in *Zig Object Notation* (`.zon`). Its location is `~/config/vesti/config.zon` on Linux and macOS, or `%APPDATA%\vesti\config.zon` on Windows. All fields are optional; the example below shows every field at its default value.

```
.{
  .engine = .tectonic,          // default LaTeX engine
  .lua = .{
    .make_log = false,         // dump failing Lua to .vesti-dummy/luacode.lua
    .line_limit = 45,          // above this many lines, log instead of print
  },
}
```

2.3 L^AT_EX engines

Vesti can target five engines. The engine may be chosen (in increasing order of precedence) by the config file, a command-line flag, the `#engine_type` builtin, or the `engine` field of `vesti.compile` in `first.lua`.

Engine	CLI flag	string id
latex	--latex / -L	"plain" or "latex"
pdflatex	--pdflatex / -p	"pdf"
xelatex	--xelatex / -x	"xe"
lualatex	--lualatex / -l	"lua"
tectonic	--tectonic / -T	"tect"

Table 1: L^AT_EX engines and how to select them

The *string id* column gives the values accepted by `#engine_type` and by `vesti.compile{ engine = ... }`. Note that `#engine_type` additionally accepts "plain" for the bare `latex` engine, whereas `vesti.compile` uses "latex".

3 The command-line interface

The `vesti` executable exposes the sub-commands below.

3.1 compile — transpile and build

```
vesti compile [ <FILENAME> ]? [ <flags> ]?
```

If `<FILENAME>` is omitted, `vesti` looks for a `first.lua` build script. The accepted flags are:

Flag	Meaning
--emit_tex, -e	emit the <code>.tex</code> only (reserved; currently a no-op)
--exit_err, -E	in watch mode, exit immediately on the first error
--no_color, -N	disable coloured terminal output
--standalone, -S	compile without assuming a <code>first.lua</code> exists
--watch, -W	rebuild when source files change
--latex, -L	use the <code>latex</code> engine
--pdflatex, -p	use <code>pdflatex</code>
--xelatex, -x	use <code>xelatex</code>
--lualatex, -l	use <code>lualatex</code>
--tectonic, -T	use <code>tectonic</code>
--lim N	number of compile cycles (default 3)
--first_script F	build-once Lua script (default <code>first.lua</code>)
--before_script F	Lua run before each pass (default <code>before.lua</code>)
--step_script F	Lua run at each step end (default <code>step.lua</code>)

Table 2: Flags of `vesti compile`

3.2 latex — build an existing .tex with Tectonic

`vesti latex <FILENAME>` compiles a plain L^AT_EX file through the bundled Tectonic engine. It accepts `--no_color`, `--watch`, `--lim` and the three script flags above.

3.3 init — scaffold a project

`vesti init <PROJECT>` creates `<PROJECT>.ves` (a tiny “Hello, World!” document) together with a `first.lua` build script wired to `vesti.compile`.

3.4 clear — remove build artefacts

`vesti clear` deletes the `.vesti-dummy` working directory.

3.5 experimental

`vesti experimental <FILENAME>` is a sandbox for in-development features and is not intended for production use.

4 Project structure and the build pipeline

A vesti build is orchestrated by up to three Lua scripts and writes its intermediate files into a hidden working directory, `.vesti-dummy`.

File	Role
<code>first.lua</code>	Build entry point, run <i>once</i> . The only place <code>vesti.compile</code> may be called.
<code>before.lua</code>	Run once <i>before each</i> compilation pass (e.g. to download fonts).
<code>step.lua</code>	Run at the <i>end of each</i> compile step (post-processing hooks).
<code>.vesti-dummy</code>	Working directory holding generated <code>.tex</code> , copied assets and downloads.

Table 3: The files that drive a vesti build

A freshly initialised `first.lua` looks like:

```

1  -- below code imports a vesti module
2  -- vesti.getModule("module_name")
3  vesti.compile("myproject.ves", { engine = "tect", compile_all = true })
```

With the `-S / --standalone` flag vesti skips the `first.lua` search and compiles the named file directly. Because compilation happens relative to `.vesti-dummy`, asset paths must be rewritten with `#get_filepath` or `#copy_file` rather than passed raw to `\includegraphics` and friends.

5 Structure of a vesti file

Like $\text{\LaTeX}$, a vesti document has two regions: the **preamble** and the **main body**, separated by `startdoc`.

```

docclass article (10pt)      -- preamble: class
importpkg {                  -- preamble: packages
  geometry (a4paper, margin = 2.2cm),
}
startdoc                     -- everything after is the body
Hello, Vesti!
```

In the preamble the keywords `docclass` and `importpkg` carry their special meaning; once `startdoc` is seen they become ordinary text, and a second `startdoc` in the body does nothing. There is no closing keyword — end-of-file terminates the document.

6 Lexical grammar

This section describes how raw characters become tokens.

6.1 Comments

Vesti comments are *Lua-style*, not $\text{\LaTeX}$ -style. A line comment begins with two hyphens and runs to the end of the line; block comments use long-bracket syntax with an optional level of = signs:

```
Hello  -- a vesti line comment, removed before output
--[[ a block comment
    spanning several lines ]]
--[==[ contains ]] and ]=] safely ]==]
```

Because two hyphens start a comment, the dashes you may want in prose have dedicated spellings: write `--!` for an en-dash and `---` for an em-dash. A percent sign `%` is *not* a vesti comment: it is passed through as a $\LaTeX$ comment.

6.2 Escapes and literal characters

Some characters are meaningful to vesti and must be escaped to appear verbatim in the output:

<code>\#</code>	<code>-></code>	<code>#</code>	literal sharp (otherwise begins a builtin)
<code>\\$</code>	<code>-></code>	<code>\$</code>	literal dollar (otherwise a math switch)
<code>\$#</code>	<code>-></code>	<code>\$</code>	alternative literal dollar
<code>\%</code>	<code>-></code>	<code>%</code>	literal percent sign
<code>#!</code>	<code>-></code>	<code>#</code>	literal sharp before an identifier (e.g. Lua's <code>#t</code>)
<code>\{ \}</code>	<code>-></code>	<code>{ }</code>	math braces
<code>\[\]</code>	<code>-></code>		display-math delimiters
<code>\,</code>	<code>-></code>		thin math space
<code>\;</code>	<code>-></code>		wide math space

6.3 Raw $\LaTeX$ pass-through (verbatim)

Two constructs hand bytes straight to $\LaTeX$, untouched by vesti's lexer. A **block** of raw $\LaTeX$ is written between `%-` and `-%`; a single **line** of raw $\LaTeX$ follows `%#`. For instance, the single-line form is used for a one-off definition such as

```
\def\thefootnote{\roman{footnote}}}
```

and the block form for a longer fragment that vesti should not touch:

```
\begin{tabular}{cc} a & b \\ c & d \end{tabular}
```

6.4 Numbers, words and $\LaTeX$ commands

Runs of digits form integers, with a decimal point producing a float. Runs of alphabetic characters form text; a text run equal to a reserved word becomes that keyword. A backslash followed by letters is a $\LaTeX$ command. When `#at_on` is active the `@` character is also a command letter, and under `#ltx3_on` so are `_` and `:` (expl3 names).

6.5 Math symbol conversion

Inside math mode many ASCII sequences are rewritten into the corresponding $\LaTeX$ commands; outside math mode they stay ordinary text. The complete table baked into the lexer:

<code><=</code>	<code>-></code>	<code>\leq</code>	<code>-></code>	<code>-></code>	<code>\rightarrow</code>
<code>>=</code>	<code>-></code>	<code>\geq</code>	<code><-</code>	<code>-></code>	<code>\leftarrow</code>
<code>!=</code>	<code>-></code>	<code>\neq</code>	<code><-></code>	<code>-></code>	<code>\leftrightarrow</code>
<code>/=</code>	<code>-></code>	<code>\neq</code>	<code>=></code>	<code>-></code>	<code>\Rightarrow</code>
<code>--></code>	<code>-></code>	<code>\longrightarrow</code>	<code><--</code>	<code>-></code>	<code>\longleftarrow</code>
<code>==></code>	<code>-></code>	<code>\Longrightarrow</code>	<code><==</code>	<code>-></code>	<code>\Longleftarrow</code>

```

<--> -> \longlefttrightarrow  <==> -> \Longlefttrightarrow
<=>   -> \Leftrightarrow      |->   -> \mapsto
-!    -> \setminus             {<    -> \langle
>}    -> \rangle              ||    -> \lvert
...    -> \cdots               oo    -> \infty
//     -> \frac{...}{...} (a fraction; see the Math mode section)

```

The `oo` sequence only becomes ∞ when it stands alone; `food` keeps its letters, because the lexer checks that no alphabetic character follows.

7 Keywords

The following words are reserved; they act as keywords only where they are syntactically valid (for example `docclass` only in the preamble) and otherwise fall back to ordinary text. The table below is generated directly from `../src/lexer/Token.zig` at compile time, so it always reflects the current `VESTI_KEYWORDS` set.

<code>beginv</code>	<code>defenv</code>	<code>defun</code>	<code>docclass</code>
<code>endenv</code>	<code>importmod</code>	<code>importpkg</code>	<code>importves</code>
<code>startdoc</code>	<code>useenv</code>		

Table 4: Keywords in Vesti (auto-generated from the source)

7.1 `docclass` — document class

The analogue of `\documentclass`; active only in the preamble.

`docclass` $\square$ `<class-name>` $\square$ (`<options>`)

Options are comma-separated and parenthesised. Examples: `docclass` $\square$ `article`, `docclass` $\square$ `article` $\square$ (10pt), and `docclass` $\square$ `article` $\square$ (10pt, $\square$ twocols).

7.2 `importpkg` — load packages

The analogue of `\usepackage`, also preamble-only. It has two forms. The single-package form mirrors `docclass`; the braced form loads several packages, each with its own options:

```

importpkg {
  amsmath, amssymb, amsthm,
  geometry (a4paper, margin = 2.2cm),
  xcolor (dvipsnames),
}

```

7.3 `importves` — include another vesti file

`importves`(`<path.ves>`) resolves the path, name-mangles it, and emits `\input{...}` of the corresponding compiled file. This lets a project be split across many `.ves` sources.

7.4 `importmod` — fetch and use a vesti module

`importmod`(`<module-name>`) downloads (if necessary) and installs an external vesti module. The Lua equivalent is `vesti.getModule`.

7.5 `startdoc` — begin the body

Marks the end of the preamble and the start of the main body (it expands to `\begin{document}`). A second `startdoc` inside the body is a no-op.

7.6 `useenv` — an environment

`useenv` replaces a `\begin{...}` / `\end{...}` pair with a single brace-delimited block; `vesti` tracks the nesting and emits the matching end for you.

```
useenv <env-name> <arg>* { <body> }
```

Each `<arg>` is either (*mandatory*) or [*optional*]; there may be zero or more, and **no spaces may separate consecutive arguments**.

```
useenv figure [ht] {
  \centering
  useenv tikzpicture {
    \draw (0,0) -- (1,1);
  }
}
```

7.7 `beginenv` / `endenv` — explicit pair

These are the explicit, unmatched analogues of `\begin` and `\end`. The block `beginenv center ... endenv` is equivalent to `useenv center { ... }`. Since `useenv` already auto-matches, why keep them? *Defining new environments*: the body of a `defenv` needs an unmatched begin in one half and an unmatched end in the other. Two modifiers may follow the keyword with no space: a `!` gives the **phantom** form (do not push/pop the matching stack), and `*` emits a trailing newline after the begin.

7.8 `defun` — define a command

`defun` defines a $\text{\LaTeX}$ command. With a parameter specification it uses the modern `xparse` interface; without one it falls back to the primitive `\def` family.

```
defun [<attr>]? <name> (<spec>)? {<body>}
```

Examples:

```
defun foo {Hello, World!}
defun [!] keyword (m) {\tt\color{vkw}\bfseries #1}}
defun [r] Re {\mathmode{\mathop{"Re"}}}
defun defeq {\coloneq}
```

The optional [`<attr>`] bracket holds single-character flags that select how the command is defined and how the body's surrounding whitespace is trimmed:

In the primitive form the body references arguments with the parameter tokens `#1` through `#9` (becoming `#1..#9`); doubled nesting is written by prefixing the index with the nesting level, so `#11` becomes `##1` and `#21` becomes `####1`. In `xparse` mode the `<spec>` is an `xparse` argument string such as `m`, `o` or `v`. To define a command whose name contains `@`, wrap the definition in `#at_on` / `#at_off`.

7.9 `defenv` — define an environment

`defenv` defines a new environment; it always uses the `xparse \NewDocumentEnvironment` family. The body is given as *two* braced blocks — the begin code and the end code:

Flag	Effect
<code>r / R</code>	redefine: do not error if the name exists
<code>!</code>	declare: use the non- <code>\protected</code> primitive / <code>\DeclareDocumentCommand</code>
<code>e / E</code>	expand: use the <code>\edef</code> family / expandable xparse
<code>g / G</code>	global: use the <code>\gdef</code> / <code>\xdef</code> family
<code>p / P</code>	provide (xparse only): <code>\ProvideDocumentCommand</code>
<code>< / ></code>	do not trim whitespace on the left / right of the body

Table 5: Attributes of `defun`

```
defenv foo {begenv! minipage(\textwidth)}{endenv!}
```

Attributes: `r/R` (redefine), `p/P` (provide), `!` (declare), `</>` keep the begin block's left/right whitespace, and `(/)` keep the end block's left/right whitespace.

8 Math mode

Vesti tracks whether it is in **text mode** or **math mode**. The symbol conversions above and the constructs below apply only in math mode.

8.1 Entering math mode

Inline math is delimited by `$...$`; display math by `$$...$$` or by `\[...\]`. A math block must be properly closed; an unbalanced switch is a parse error.

8.2 Fractions with `//`

Inside a brace group in math mode, the `//` operator splits the group into a numerator and a denominator and emits `\frac`:

```
$$ {1 // n^2} $$      -- becomes \frac{1}{n^2}
$$ {\pi^2 // 6} $$    -- becomes \frac{\pi^2}{6}
```

8.3 Auto-sized delimiters with `?`

A `?` immediately *before* an opening delimiter, or immediately *after* a closing delimiter, requests an auto-sized (`\left/\right`) delimiter. The recognised delimiters are:

```
( )    [ ]    {< >}    \{ \}    |    ||    .
```

(a lone dot yields an invisible delimiter).

```
$ ?( \frac{a}{b} )? $      -- \left( \frac{a}{b} \right)
```

8.4 Text inside math with `"..."`

Double quotes drop back into text mode for their contents (like `\text{}`). Prefixing or suffixing the quotes with a raw sharp `#` adds a surrounding space:

```
$ x = "if " n " is even" $ -- text spans, no auto spaces
```

```
$ a #"and"# b $          -- a space before and after the word
```

8.5 Mode-switching builtins

#mathmode forces math interpretation of its braced body (and errors if already in math mode); **#textmode** does the reverse. They are indispensable when a command body must be parsed in a particular mode regardless of where the command is later used.

9 Builtins

Builtins are compiler intrinsics, written with a leading **#**. The prefix exists so that common English words (like *label*) are not stolen from ordinary prose. The **evaluated** builtins below are generated from VESTI_BUILTINS in `../src/lexer/Token.zig`:

#chardef	#copy_file	#engine_type	#enum	#enum_counter
#eq	#get_filepath	#label	#mathchardef	#mathmode
#picture	#raw_tex	#showfont	#textmode	

Table 6: Evaluated builtins (auto-generated from the source)

The **preprocessor** builtins (resolved in an earlier pass; see the next section) come from VESTI_PREPROCESS_BUILTINS in the same file:

#at_off	#at_on	#def	#include
#ltx3_off	#ltx3_on	#noltx3	#undef

Table 7: Preprocessor builtins (auto-generated from the source)

Since **fancyvrb** does not allow verbatim-like commands inside the **Verbatim** environment, builtins are not coloured inside example code blocks.

9.1 #label

`\label` normally has to go *inside* an environment body, which reads awkwardly. **#label** attaches the label to the **useenv** that follows it instead; it must immediately precede a **useenv**.

```
#label(eq:1)
useenv theorem {
  1 + 1 = 2.
}
```

9.2 #eq

A shorthand for an **equation** environment (optionally labelled). It must be used in text mode and switches to math mode for its body:

```
#eq(eq:euler) {
  \sum_{n=1}^{\infty} {1 // n^2} = {\pi^2 // 6}.
}
```

9.3 #chardef

#chardef `␣` `<hex-codepoint>` `␣` `<command>` defines a text character token by Unicode code point (hexadecimal), emitting a `\chardef`. The directive must end at a newline.

9.4 #mathchardef

#mathchardef $\square$ $\langle$ class $\rangle$ $\square$ $\langle$ font-num $\rangle$ $\square$ $\langle$ hex $\rangle$ $\square$ $\langle$ command $\rangle$ defines a math symbol by code point and *math class*, emitting a `\Umathchardef`. The class is one of `.ordinary(0)`, `.largeop(1)`, `.binary(2)`, `.relation(3)`, `.opening(4)`, `.closing(5)`, `.punct(6)` or `.variable(7)`.

```
#mathchardef .opening 0 29d8 \lfoo
#mathchardef .ordinary 0 2202 \diff
```

9.5 #enum and #enum_counter

#enum is a compact `enumerate` with light `enumitem`-style label control; it nests up to four deep without `enumitem`. An optional parenthesised pattern customises the `\item` label: a single `*` expands to the current counter $\{\langle$ count $\rangle\}$ (braces included), while a doubled `**` collapses to one literal `*`.

```
#enum (\Roman*. ) {
  \item aaa
  \item #enum ({\roman*}**) {
    \setcounter{#enum_counter}{2}
    \item bbb
  }
}
```

Here `\Roman*.` becomes `\Roman{enumi}.` and `{\roman*}**` becomes `{\roman{enumi}}*`. The companion **#enum_counter** expands to the *name* of the current enum counter, useful with `\setcounter`.

9.6 #get_filepath

Because compilation runs inside `.vesti-dummy`, raw relative paths break. **#get_filepath**($\langle$ path $\rangle$) rewrites a path so it is correct relative to the dummy directory:

```
\includegraphics{#get_filepath(./foo.png)}
```

A leading `~` expands to the home directory and a leading `@/` anchors the path at the vesti config directory.

9.7 #copy_file

#copy_file($\langle$ path $\rangle$) copies the named file into `.vesti-dummy` so the engine can find it. Like **#get_filepath** it understands the `~` and `@/` prefixes. It replaces the deprecated `cpfile` keyword.

9.8 #picture

#picture $\square$ [$\langle$ unit $\rangle$]? $\square$ (w , h) $\square$ (x , y)? $\square$ $\{\langle$ body $\rangle\}$ generates a $\text{\LaTeX}$ `picture` environment, where w , h are integer width and height, the optional second pair is an offset, and the optional bracketed argument sets the unit length.

9.9 #showfont

#showfont(n) expands to a `\ttfamily\meaning\textfontn` report for math font number n (0–255). A diagnostic aid.

9.10 `#raw_tex`

Placed directly before a `defun`, `#raw_tex` disables the automatic xparse machinery so the definition uses the bare `\def` primitive even when a parameter list is present.

9.11 `#engine_type`

`#engine_type`(`<id>`) selects the L^AT_EX engine from within the source. It must appear on the very first line and may be used only once; `<id>` is one of `plain`, `pdf`, `xe`, `lua` or `tect`. It replaces the deprecated `compty` keyword.

9.12 `#textmode` and `#mathmode`

Discussed under Math mode above. `#mathmode`{`<body>`} forces math interpretation of its body; `#textmode`{`<body>`} forces text interpretation (valid only inside math mode).

10 The preprocessor

Before parsing, vesti runs a token-level preprocessing pass that expands user-defined macros and resolves the eight directives listed in the preprocessor table above.

10.1 `#def` and `#undef` — token macros

`#def` `␣` `#name` `␣` {`<body>`} defines a preprocessor macro `#name`. Wherever `#name` later appears it is replaced by the body, with expansion applied recursively. Macros may take parameters, referenced in the body as `#1`, `#2`, ...; arguments are supplied in parentheses at the call site.

```
#def #greet {Hello, World!}
#def #twice {#1 #1}

#greet           -- expands to: Hello, World!
#twice(ho)       -- expands to: ho ho
#undef #twice
```

A parameter count is inferred from the highest parameter used in the body. You cannot `#def` or `#undef` a name that is already a builtin or keyword, and preprocessor directives may not appear inside a macro body.

10.2 `#include` — textual file inclusion

`#include`(`<path>`) splices the *token stream* of another file into the current one at that point (distinct from `importves`, which emits an `\input`). Macros defined in the included file therefore become visible to the includer. Circular includes are detected and rejected.

10.3 `#at_on` / `#at_off` — @ as a letter

`#at_on` emits `\makeatletter` and tells the lexer to treat `@` as a command-name letter; `#at_off` emits `\makeatother` and restores the lexer. This is required to `defun` commands whose names contain `@`:

```
#at_on
defun @foo (#1) {Hello, #1!}
#at_off
```

10.4 #ltx3_on / #ltx3_off / #noltx3

`#ltx3_on` emits `\ExplSyntaxOn` and makes `_` and `:` command letters (so you can write `expl3` names); `#ltx3_off` emits `\ExplSyntaxOff`. Since the modern L^AT_EX kernel permits `expl3` without a separate package, these are enabled by default. The preamble-only `#noltx3` opts out, forbidding the other two for that file.

10.5 Defun parameter tokens

The sharp-number tokens `#1` through `#9` act as parameter placeholders in both `#def` bodies and `defun` bodies. The two-digit forms encode nesting for definitions-within-definitions: the tens digit is the nesting level and the units digit the argument index, and the emitted number of `#` signs doubles with each level.

11 Lua scripting

Vesti embeds a full Lua interpreter, used in the build scripts and in inline blocks. All of Lua’s standard library is available, plus a global `vesti` table of compiler bindings.

11.1 Inline Lua blocks

An inline block is delimited by `#::` and `::#`. The code runs at compile time; whatever it sends through `vesti.print` (or `vesti.parse`) is spliced into the document at that position.

```
#::
for i = 1, 3 do
    vesti.print("\item entry " .. i)
end
::#
```

Note: `vesti.print` inserts its argument as *raw* L^AT_EX, while `vesti.parse` first transpiles a string of *vesti* into L^AT_EX.

11.2 Block modifiers: global, import, export

Three optional modifiers may follow the closing `::#`, in this order:

Suffix	Meaning
<code>::#*</code>	global : definitions persist in the shared Lua state
<code>::#[a, b]</code>	import : pull the exported chunks <code>a</code> and <code>b</code> into scope
<code>::#<name></code>	export : register this block’s source under <code>name</code>

Table 8: Inline Lua block modifiers

They compose, e.g. `::#[helpers]<more>` runs globally, imports `helpers`, and exports itself as `more`. A common pattern is to export a helper block once and import it wherever needed:

```
#::
local function read_all(path)
    local f = assert(io.open(path, "rb"))
    local data = f:read("*a"); f:close(); return data
end
::#<readAll>

#::
local content = read_all("data.txt")
vesti.print(content)
::#[readAll]
```

Because the lexer still scans inline blocks, a bare `#` before an identifier (such as Lua’s length operator `#t`) would be read as a builtin. Write `#!t` — the `#!` escape yields a literal `#` — when you need `#` immediately before a name.

11.3 Build scripts

`first.lua` runs once at the very start and must call `vesti.compile`; `before.lua` runs once before each compilation pass; `step.lua` runs at the end of each step. A representative `before.lua` that downloads a font:

```
1 local dummy = vesti.vestiDummyDir()
2 local url = "https://example.com/STIXTwoMath-Regular.ttf"
3 vesti.mkdir(dummy)
4 vesti.download(url, vesti.joinpath(dummy, "STIXTwoMath-Regular.ttf"))
```

11.4 vesti.compile — the build entry point

Callable *only* from `first.lua`: `vesti.compile(<main-ves>, { <options> })`. The options table accepts the string `engine` ("latex", "pdf", "xe", "lua" or "tect") and the booleans `compile_all`, `watch`, `no_color` and `no_exit_err`.

```
1 vesti.compile("paper.ves", { engine = "tect", compile_all = true })
```

12 The Lua API reference

Every binding lives in the global `vesti` table. The list below is generated from `../src/Lua.zig` so it always reflects the functions the compiler actually exposes:

compile	download	getCurrentDir	getEngineType
getModule	joinpath	mkdir	parse
ping	print	setCurrentDir	unzip
vestiDummyDir			

Table 9: Lua API functions in Vesti (auto-generated from the source)

What each function does:

Function	Purpose
<code>print(..., {sep, nl})</code>	append arguments to the output (raw $\text{\LaTeX}$); <code>sep</code> default " ", <code>nl</code> default 1, clamped 0–2
<code>parse(s)</code>	transpile a vesti string to $\text{\LaTeX}$ and return it
<code>mkdir(d)</code>	create a directory (silent if it exists)
<code>joinpath(...)</code>	join path components, return the path
<code>getCurrentDir()</code>	return the current working directory
<code>setCurrentDir(d)</code>	change the current working directory
<code>vestiDummyDir()</code>	return the <code>.vesti-dummy</code> directory name
<code>unzip(file, dest)</code>	extract a zip into an existing directory
<code>download(url, file)</code>	HTTP GET into a file
<code>ping(url)</code>	return <code>true</code> if the host is reachable
<code>getModule(name)</code>	download and install a vesti module
<code>getEngineType()</code>	return the active engine as a string
<code>compile(ves, {opts})</code>	build entry point (<code>first.lua</code> only)

Table 10: Behaviour of the Lua API functions

13 Deprecated features

The following keywords have been retired in favour of builtins; using them is a compile error that names the replacement. This table is generated from the deprecated entries in VESTI_KEYWORDS:

Deprecated	Replacement
<code>compty</code>	<code>#engine_type</code>
<code>cpfile</code>	<code>#copy_file</code>

Table 11: Deprecated keywords (auto-generated from the source)

14 A worked example

A small but complete document exercising several features at once:

```
docclass article (11pt)
importpkg {
    amsmath, amssymb,
    geometry (a4paper, margin = 2.5cm),
    graphicx,
}

defun [!] R {\mathbb{R}}           -- a quick math macro
#def #vec {\mathbf{#1}}           -- a parameterised preprocessor macro

startdoc
\section{A demonstration}

The reals are written  $\$#vec(x)$  in  $R^n$ , and we recall
#eq(eq:zeta) {
    \sum_{n=1}^{\infty} 1 // n^2 = {\pi^2 // 6}.
}

#enum (\Roman*. ) {
    \item first
    \item second
}

\includegraphics[width=0.4\textwidth]{#get_filepath(./figure.png)}

#::
for _, name in ipairs({ "alpha", "beta", "gamma" }) do
    vesti.print(vesti.parse("\textbf{" .. name .. "}\\\\"))
end
::#
```

15 Appendix: quick reference

Auto-generated tables. The keyword, evaluated-builtin, preprocessor-builtin, Lua-API and deprecated tables in this manual are produced by inline Lua that reads `../src/lexer/Token.zig` and `../src/Lua.zig`. Editing the corresponding maps in the compiler source and recompiling this document updates every list automatically — no manual edits required.

Syntax at a glance.

--	line comment	--[[]]	block comment
--!	en-dash	---	em-dash
%#	line verbatim	%-...-%	block verbatim
\$...\$	inline math	\$\$...\$\$	display math
{a // b}	fraction (math)	? ()?	sized delimiter
" ..."	text in math	oo	infinity (math)
#1 ...#9	defun parameter	#!	literal #
#:: ...::#	inline Lua	::#*	global Lua block

End of the vesti full reference manual.

Lists are generated from a live reading of the compiler source.